Projet de session - Déploiement d'une application multi-conteneur sur un Cluster Docker Swarm ou Kubernetes

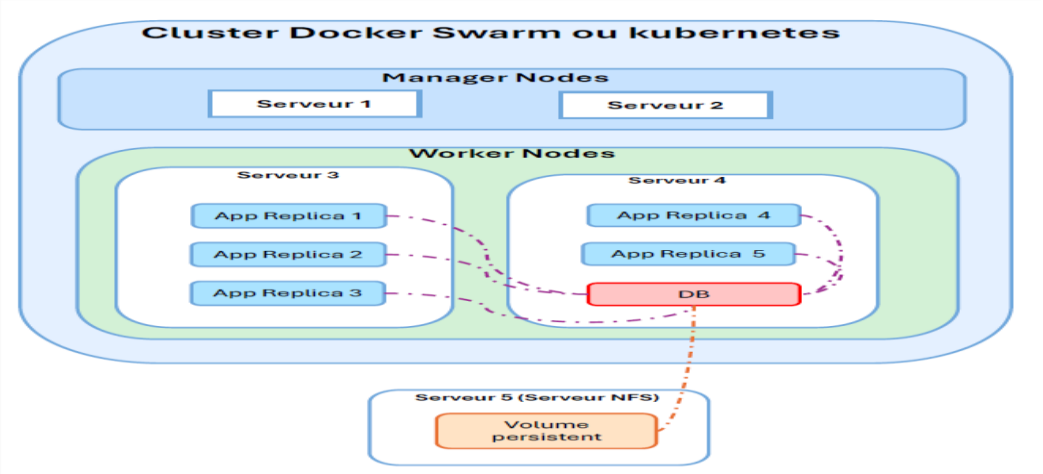
Membres du groupe :

- **Taylor Ngue Ngan** (Manager VMLinux128)
- **Iboune Oumar Balde** (Manger VMLinux108)
- **Hanane Zerrouki** (Serveur NFS VMLinux129)
- **Hicham Hamdi** (Worker VMLinux221)
- **Massinissa Makoudi** (Worker VMLinux226)
- **Nour Miri** (Worker VMLinux130)

Introduction

Ce projet a pour objectif de déployer une application multi-conteneur sur un cluster Docker Swarm afin de démontrer les principes de l'orchestration des conteneurs.
L'architecture mise en place repose sur plusieurs nœuds managers et workers, assurant la haute disponibilité et la répartition de charge.
Un serveur NFS est utilisé pour garantir la persistance des données de la base de données.
Le projet permet également de valider les mécanismes de scaling, de self-healing et de failover offerts par Docker Swarm.

Architecture du projet



I) Mises en place de l'infrastructures

i) Mises en place du cluster

L'initialisation réussie d'un cluster Docker Swarm sur un serveur Linux VM128 considéré comme le manager principale.
La commande suivante est exécutée : **docker swarm init --advertise-addr 10.7.237.117** pour les workers et **docker swarn join-token manager** pour le manager secondaire.

```
admin@VMLinux128:~$ docker swarm init --advertise-addr 10.7.237.117
Swarm initialized: current node (qxx9ep8h0ibz0tbsn2h98v93k) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-1rfd3e3umdlzchjipu7cw3a8xe3wul6ykldwjnoi26vbu0nw93-1m3qdd98iwz8s91a0ene0ra6l 10.7.237.117:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
admin@VMLinux128:~$ ^C
admin@VMLinux128:~$ ^C
admin@VMLinux128:~$ docker swarm join-token manager
To add a manager to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-1rfd3e3umdlzchjipu7cw3a8xe3wul6ykldwjnoi26vbu0nw93-37iqsglctjlcnx6gam6pdaect 10.7.237.117:2377

admin@VMLinux128:~$ ^C
admin@VMLinux128:~$
```

Commande exécutée chez le manager secondaire et les différents workers.

Visualisation du cluster : **docker node ls**

```
ID                            HOSTNAME     STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
yn4tbwlzqeq21k6e9zknd27cf     VMLinux108   Ready     Active         Reachable        29.3.1
pqf9sig6u4qtqtr8bb59dubq6 *   VMLinux128   Ready     Active         Leader           29.3.1
s630mcxo5f7sjqaq6lqoxwuc1     VMLinux130   Ready     Active                          29.3.0
qelfousiwa9pq3gabw2s0vdum     VMLinux221   Ready     Active                          29.3.0
vxx5exdhmxvawzxhytgfcd7um     VMLinux226   Ready     Active                          29.3.0
admin@VMLinux128:~$
```

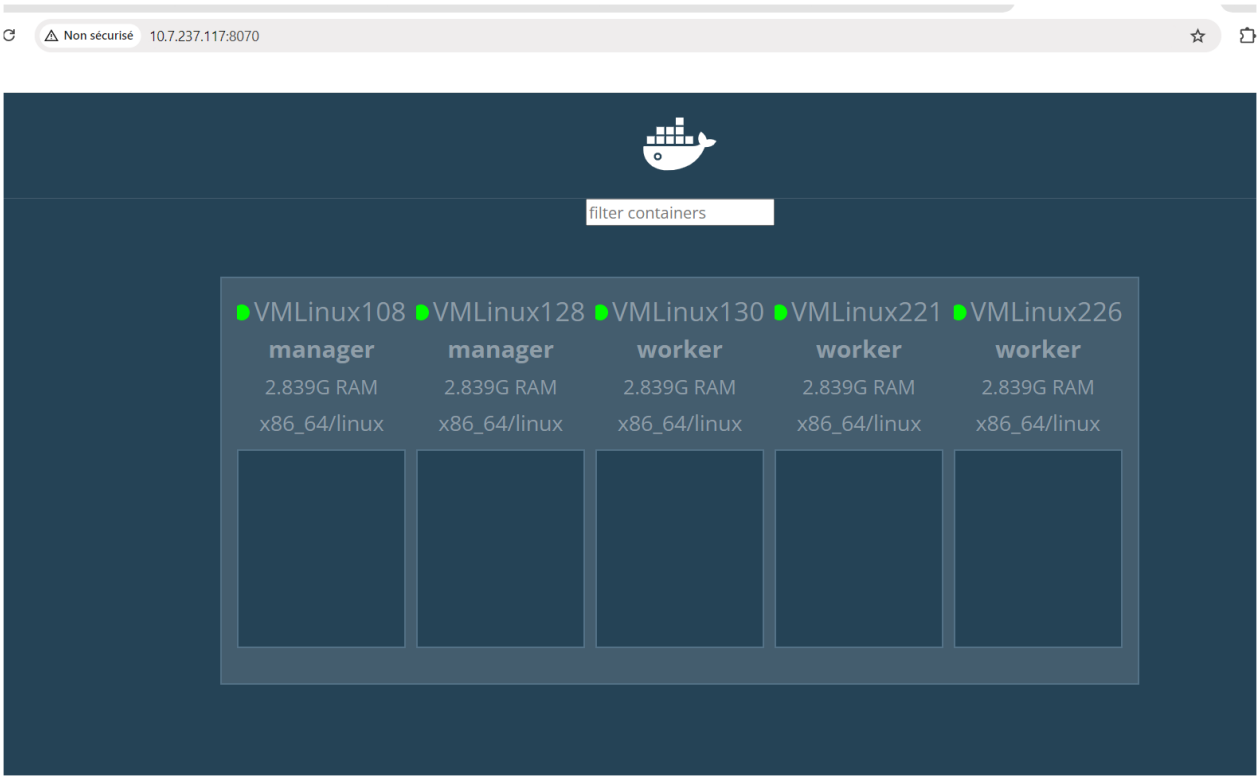
ii) Outil de visualisation graphique du cluster

Exécution de la commande sur un nœud du cluster Docker Swarm et mappé au port 8070

Pour l'exercice nous l'avons effectué dans la VM128.

```
admin@VMLinux128:~$ docker run -it -d -p 8070:8080 -v /var/run/docker.sock:/var/run/docker.sock dockersamples/visualizer
Unable to find image 'dockersamples/visualizer:latest' locally
latest: Pulling from dockersamples/visualizer
021a328e5f7b: Pull complete
ddad3d7c1e96: Pull complete
3a8370f05d5d: Pull complete
71a8563b7fea: Pull complete
119c7e14957d: Pull complete
28bdf67d9c0d: Pull complete
12571b9c0c9e: Pull complete
e1bd03793962: Pull complete
3ab99c5ebb8e: Pull complete
94993ebc295c: Pull complete
Digest: sha256:530c863672e7830d7560483df66beb4cbbcd375a9f3ec174ff5376616686a619
Status: Downloaded newer image for dockersamples/visualizer:latest
facff3538357fc701a9dc5fc6eefb21fea6f0088e5896ed0bf055a36272a3d38
admin@VMLinux128:~$
```

Pour la vérification, il faut aller dans un navigateur et taper **http://10.7.237.117:8070**



iii) Configuration du serveur NFS et validation du montage NFS sur tous les nœuds

- Configuration serveur NFS dans la VM129

```
admin@VMLinux129: $ sudo apt update
[sudo] password for admin:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://ca.archive.ubuntu.com/ubuntu noble InRelease
Get:3 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Hit:4 http://ca.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://ca.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:6 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [49.7 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,170 kB]
Fetched 1,395 kB in 1s (965 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
28 packages can be upgraded. Run 'apt list --upgradable' to see them.
admin@VMLinux129: $ sudo apt install nfs-kernel-server -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  keyutils libnfsidmap1 nfs-common rpcbind
Suggested packages:
  watchdog
The following NEW packages will be installed:
  keyutils libnfsidmap1 nfs-common nfs-kernel-server rpcbind
0 upgraded, 5 newly installed, 0 to remove and 28 not upgraded.
Need to get 569 kB of archives.
After this operation, 2,022 kB of additional disk space will be used.
Get:1 http://ca.archive.ubuntu.com/ubuntu noble-updates/main amd64 libnfsidmap1 amd64 1:2.6.4-3ubuntu5.1 [48.3 kB]
Get:2 http://ca.archive.ubuntu.com/ubuntu noble/main amd64 rpcbind amd64 1.2.6-7ubuntu2 [46.5 kB]
Get:3 http://ca.archive.ubuntu.com/ubuntu noble/main amd64 keyutils amd64 1.6.3-3build1 [56.8 kB]
Get:4 http://ca.archive.ubuntu.com/ubuntu noble-updates/main amd64 nfs-common amd64 1:2.6.4-3ubuntu5.1 [248 kB]
Get:5 http://ca.archive.ubuntu.com/ubuntu noble-updates/main amd64 nfs-kernel-server amd64 1:2.6.4-3ubuntu5.1 [169 kB]
Fetched 569 kB in 1s (1,037 kB/s)
```

Montage persistant dans le fichier de configuration nfs pour le répertoire partagé data
Commande **: mkdir -p /srv/nfs/data (**pour créer le répertoire partagé)

Commande : **sudo nano /etc/exports**

```
admin@VMLinux129: ~
  GNU nano 7.2                                                    /etc/exports
# /etc/exports: the access control list for filesystems which may be exported
#               to NFS clients.  See exports(5).
#
# Example for NFSv2 and NFSv3:
# /srv/homes       hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
#
# Example for NFSv4:
# /srv/nfs4        gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
# /srv/nfs4/homes  gss/krb5i(rw,sync,no_subtree_check)
#
/srv/nfs/data  10.7.236.0/16(rw,sync,no_subtree_check,no_root_squash)
```

Modification des droits dans le répertoire partagé pour permettre aux autres éditer

```
admin@VMLinux129: $ sudo chown -R nobody:nogroup /srv/nfs/data
admin@VMLinux129: $ sudo chmod -R 777 /srv/nfs/data
```

- Configuration du NFS client dans chaque nœud
  Commande : **mkdir -p /mnt/nfs (**pour créer le repertoire)

```
/srv/nfs/data 10.7.236.0/16
admin@VMLinux128:/$ sudo mount 10.7.236.115:/srv/nfs/data /mnt/nfs
admin@VMLinux128:/$ dh -kh
Command 'dh' not found, but can be installed with:
sudo apt install debhelper
admin@VMLinux128:/$ df -kh
Filesystem                         Size  Used Avail Use% Mounted on
tmpfs                              291M  1.3M  290M   1% /run
efivarfs                           128M   26K  128M   1% /sys/firmware/efi/efivars
/dev/mapper/ubuntu--vg-ubuntu--lv   23G  9.7G   13G  45% /
tmpfs                              1.5G     0  1.5G   0% /dev/shm
tmpfs                              5.0M     0  5.0M   0% /run/lock
/dev/sda2                          2.0G  201M  1.6G  11% /boot
/dev/sda1                          1.1G  6.2M  1.1G   1% /boot/efi
tmpfs                              291M   12K  291M   1% /run/user/1001
10.7.236.115:/srv/nfs/data          23G  7.2G   15G  33% /mnt/nfs
admin@VMLinux128:/$
```

- Montage persistant dans le fichier de configuration nfs client pour le répertoire partagé

```
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point>   <type>  <options>       <dump>  <pass>
# / was on /dev/ubuntu-vg/ubuntu-lv during curtin installation
/dev/disk/by-id/dm-uuid-LVM-CcDGnxxOjzsXDf1cTFZyKwsxR2iF7TWuZJWHe9Qzw74z4FSmlPK9ScYRAmqyfwz4 / ext4 defaults 0 1
# /boot was on /dev/sda2 during curtin installation
/dev/disk/by-uuid/b11c3097-6eab-451d-acdc-6b89f2cdc937 /boot ext4 defaults 0 1
# /boot/efi was on /dev/sda1 during curtin installation
/dev/disk/by-uuid/B9E2-233B /boot/efi vfat defaults 0 1
/swap.img       none    swap    sw      0       0
10.7.236.115:/srv/nfs/data /mnt/nfs nfs defaults 0 1
```

II) Déploiement

i) Création du réseau overlay qui est le réseau nécessaire au docker Swarn

Commande: **docker network create –driver overlay app_net**

```
09fe9046dac2   tp11_wp_network   bridge    local
admin@VMLinux128:~$ docker network create --driver overlay app_net
1nud5h0h2jkz5wlnmp1lk3yke
admin@VMLinux128:~$ docker network ls
NETWORK ID     NAME              DRIVER    SCOPE
1nud5h0h2jkz   app_net           overlay   swarm
9e7010b856fc   bridge            bridge    local
0acf748ff9c5   docker_gwbridge   bridge    local
fefd0dd7d7df   host              host      local
bmumrmdo5g8q   ingress           overlay   swarm
fb6593e3c8f7   none              null      local
09fe9046dac2   tp11_wp_network   bridge    local
admin@VMLinux128:~$
```

- Scrip docker-stack.yml

```yaml
version: '3.8'

services:
  app:
    image: nginx:latest
    ports:
      - "92:80"
    volumes:
      - type: bind
        source: /mnt/nfs/app
        target: /usr/share/nginx/html
    networks:
      - app_net
    deploy:
      replicas: 3
      restart_policy:
        condition: any
      placement:
        constraints:
          - node.role == worker   # ☑ seulement sur les workers

  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: appdb
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - app_net
    deploy:
      replicas: 1
      restart_policy:
        condition: any
      placement:
        constraints:
```

```
  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: appdb
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - app_net
    deploy:
      replicas: 1
      restart_policy:
        condition: any
      placement:
        constraints:
          - node.role == worker   # ☑ seulement sur les workers

networks:
  app_net:
    external: true
    name: app_net

volumes:
  db_data:
```

ii) Déploiement de notre application multi-services (App + DB)

Commande: **docker stack deploy -c ~/docker-stack.yml app_stack**

```
admin@VMLinux128:~$ docker stack deploy -c ~/docker-stack.yml app_stack
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network app_stack_app_net
Creating service app_stack_app
Creating service app_stack_db
admin@VMLinux128:~$
```

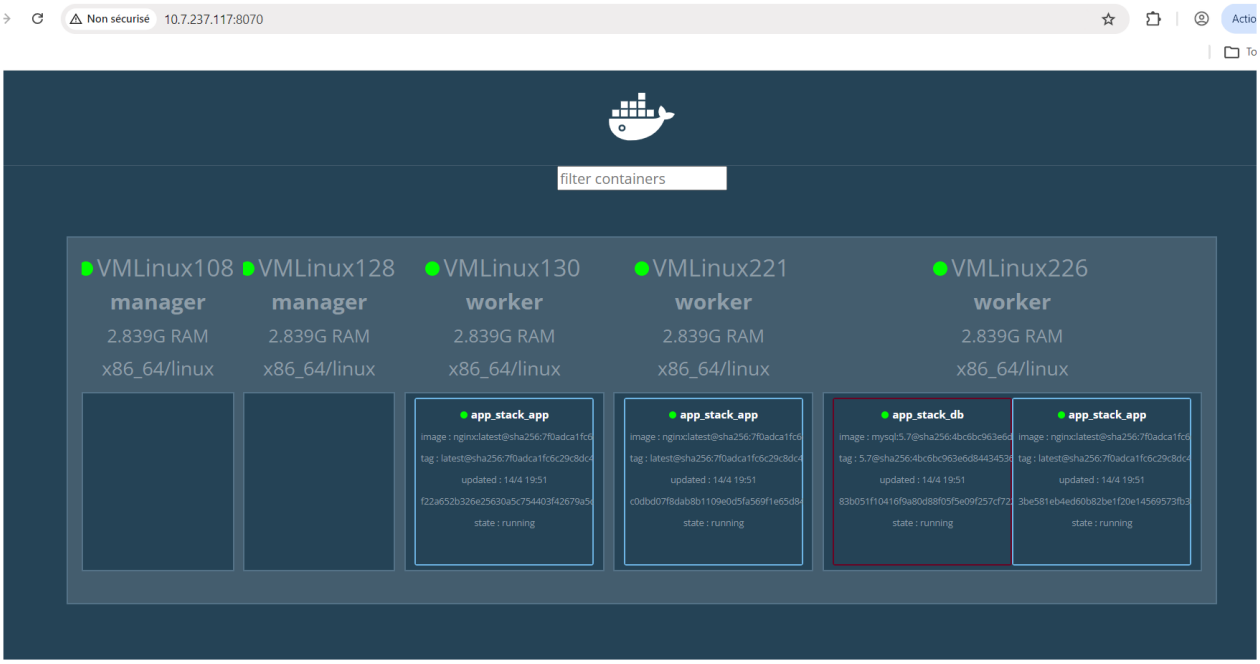
iii) Illustration des services deployé:

En ligne de commande: **docker stack services app_stack**

docker stack ps app_stack

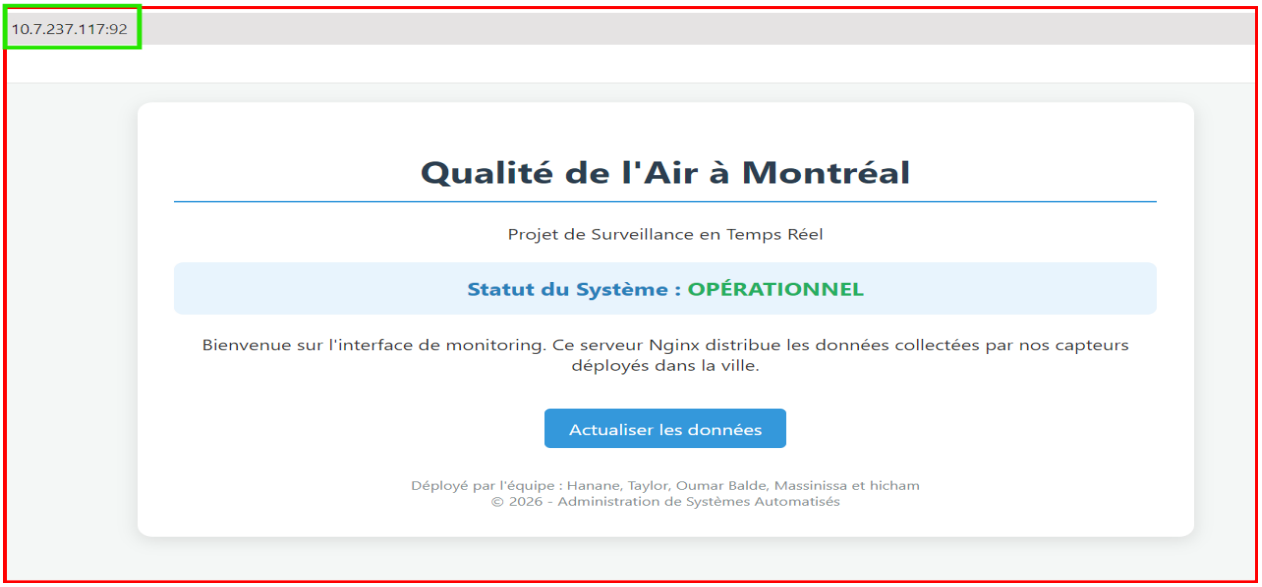
```
admin@VMLinux108:~$ docker stack services app_stack
ID             NAME            MODE         REPLICAS   IMAGE          PORTS
rz9pinbjtety   app_stack_app   replicated   3/3        nginx:latest   *:92->80/tcp
j4htqe8onu3k   app_stack_db    replicated   1/1        mysql:5.7
admin@VMLinux108:~$
```

Grafiquement:



Test de l'application sur tous les nœuds

Le site est accessible sur tous les nœuds en utilisant le port 92



iv) Test de la mise en place la persistance des données (NFS)

Commande: **docker service inspect app_stack_db**

```
admin@VMLinux128:~$ docker service inspect app_stack_db
[
    {
        "ID": "j4htqe8onu3kkvplmll43ji1q",
        "Version": {
            "Index": 46
        },
        "CreatedAt": "2026-04-13T19:43:07.742765268Z",
        "UpdatedAt": "2026-04-13T19:43:07.750201617Z",
        "Spec": {
            "Name": "app_stack_db",
            "Labels": {
                "com.docker.stack.image": "mysql:5.7",
                "com.docker.stack.namespace": "app_stack"
            },
            "TaskTemplate": {
                "ContainerSpec": {
                    "Image": "mysql:5.7@sha256:4bc6bc963e6d8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb",
                    "Labels": {
                        "com.docker.stack.namespace": "app_stack"
                    },
                    "Env": [
                        "MYSQL_DATABASE=appdb",
                        "MYSQL_ROOT_PASSWORD=root"
                    ],
                    "Privileges": {
                        "CredentialSpec": null,
                        "SELinuxContext": null,
                        "NoNewPrivileges": false
                    },
                    "Mounts": [
                        {
                            "Type": "volume",
                            "Source": "app_stack_db_data",
                            "Target": "/var/lib/mysql",
                            "VolumeOptions": {
                                "Labels": {
                                    "com.docker.stack.namespace": "app_stack"
                                }
                            }
```

-Redémarrage du conteneur pour vérifier la persistance

Commande: **docker service rm app_stack_db | docker stack deploy -c ~/docker-stack.yml app_stack**

La persistance des données est assurée par un serveur NFS centralisé.
Le service de base de données utilise un volume NFS partagé entre tous les nœuds du cluster, ce qui permet de conserver les données même en cas de redémarrage du conteneur ou de défaillance d'un nœud.

v) Connexion à la base de données pour coder les tables

Sur le worker qui a la base de données exécutons ces commandes :

Docker ps
docker exec -it CONTAINER_DB mysql -u root -p

```
admin@VMLinux226:~$ docker exec -it c1e317e90cf3 mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 2
Server version: 5.7.44 MySQL Community Server (GPL)

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> |
```

SHOW DATABASES;

```
mysql>
mysql>
mysql>
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| appdb              |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.00 sec)

mysql> |
```

Pour utiliser la base de données **(USE appdb;)**
Pour voir les tables **(SHOW TABLES;)**
Pour créer la table user **(CREATE TABLE users (**
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100),
email VARCHAR(100)
);)

vi) Pour insérer les données
INSERT INTO users (name, email)
VALUES ('Ali', 'ali@email.com');

vii) Pour voir la table
SELECT * FROM users;

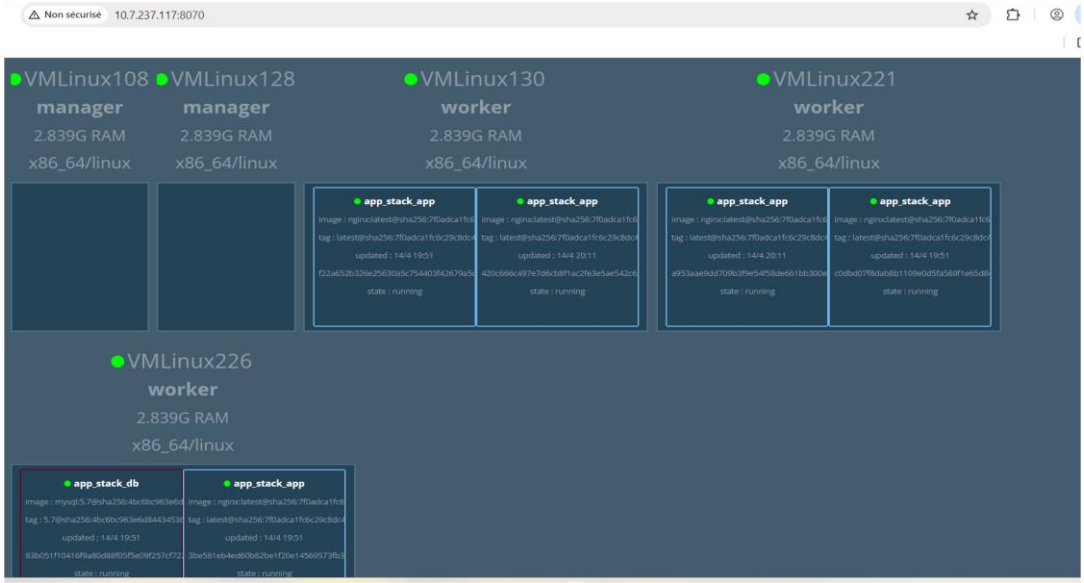
```
mysql> SELECT * FROM users;
+----+------------+----------------------+
| id | name       | email                |
+----+------------+----------------------+
|  1 | Taylor     | ali@email.com        |
|  2 | Bal        | Bal@email.com        |
|  3 | Hanane     | Hanane@email.com     |
|  4 | Massinissa | massinissa@email.com |
+----+------------+----------------------+
4 rows in set (0.00 sec)

mysql> |
```

III) Tests d'orchestration

1) Scalabilité (scaling)

Commande: **docker service scale app_stack_app=5**




```
admin@VMLinux128:~$ docker service scale app_stack_app=5
app_stack_app scaled to 5
overall progress: 5 out of 5 tasks
1/5: running   [==================================================>]
2/5: running   [==================================================>]
3/5: running   [==================================================>]
4/5: running   [==================================================>]
5/5: running   [==================================================>]
verify: Service app_stack_app converged
admin@VMLinux128:~$ docker service ls
ID             NAME           MODE         REPLICAS   IMAGE          PORTS
c7uvryd6blwr   app_stack_app  replicated   5/5        nginx:latest   *:92->80/tcp
lguuhwfvhlx8   app_stack_db   replicated   1/1        mysql:5.7
admin@VMLinux128:~$
```

2) Self-healing

Le mécanisme de self-healing de Docker Swarm permet de garantir la disponibilité d'un service en redémarrant automatiquement un conteneur arrêté ou défaillant

Stoppons un conteneur dans le cluster
Commande **: docker ps**
docker stop CONTAINER_ID

```
admin@VMLinux226:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS
83b051f10416   mysql:5.7      "docker-entrypoint.s…"   25 minutes ago   Up 25 minutes   3306/tcp, 33060/tcp
yu8qnals5dwf0d
3be581eb4ed6   nginx:latest   "/docker-entrypoint.…"   25 minutes ago   Up 25 minutes   80/tcp
7kg2o2jvaztuuln
14c03447a803   mon-nginx      "/docker-entrypoint.…"   2 weeks ago      Up 2 weeks      0.0.0.0:8090->80/tcp, [::]:8090->80/tcp
admin@VMLinux226:~$ docker stop 3be581eb4ed6
3be581eb4ed6
admin@VMLinux226:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS
1a5f3d274aa5   nginx:latest   "/docker-entrypoint.…"   2 minutes ago    Up 2 minutes    80/tcp
bb983u5r3jy1jsr
83b051f10416   mysql:5.7      "docker-entrypoint.s…"   29 minutes ago   Up 29 minutes   3306/tcp, 33060/tcp
yu8qnals5dwf0d
14c03447a803   mon-nginx      "/docker-entrypoint.…"   2 weeks ago      Up 2 weeks      0.0.0.0:8090->80/tcp, [::]:8090->80/tcp
admin@VMLinux226:~$
```

3) Failover dans un worker

Commande : **docker node update --availability drain VMLinux130**

Conclusion

Ce projet a permis de déployer une application multi-conteneur sur un cluster Docker Swarm en respectant une architecture distribuée avec stockage persistant via NFS. Les tests réalisés ont validé la scalabilité, le self-healing et le failover, démontrant la robustesse et la haute disponibilité de la solution. Ce travail a renforcé la compréhension pratique des mécanismes d'orchestration et des architectures cloud-native modernes.